CYBER SECURITY INTERNSHIP

TASK 3

# API Security Risk Analysis Report

*A read-only security assessment of a public REST API using Postman*

Prepared by

**Pavan Kumar Reddy Putta**

**Future Interns**

July 2026

# 1. Introduction

Application Programming Interfaces (APIs) are the connective layer of modern software. They allow separate applications to exchange data over HTTP by sending structured requests and receiving structured responses, and they sit behind almost every web platform, mobile application, SaaS product, and cloud service in use today.

Because APIs routinely carry business logic and sensitive user data, they have become one of the most frequently targeted components of an application. Weak authentication, missing authorization checks, excessive data exposure, and insecure configuration are recurring causes of real-world breaches, and they are difficult to detect from the outside without deliberate inspection.

This report documents a read-only security analysis of a publicly available demonstration API. The goal was to observe how the API behaves, examine what data and metadata it returns, and identify the security risks those observations would represent in a production environment. No exploitation, bypass, or destructive testing was performed at any stage; every request was a standard, non-intrusive read operation.

## 2. Objectives of the Assessment

The assessment was carried out with the following objectives in mind:

1. Understand how REST APIs communicate through HTTP requests and responses.
2. Examine publicly accessible API endpoints using Postman.
3. Analyse request methods, response payloads, and HTTP response headers.
4. Verify whether authentication is required before API resources can be accessed.
5. Identify potential API security risks based on the observed behaviour.
6. Assess the business impact those risks would carry in a production setting.
7. Recommend improvements aligned with recognised API security best practices.
8. Document the complete assessment in a clear, professional security report.

## 3. Scope of the Assessment

### Target API

JSONPlaceholder — a free, public REST API created for developers to practice API testing and application development.

**Base URL:** https://jsonplaceholder.typicode.com

### Endpoints Examined

- /posts — collection of post records
- /users — user profile records
- /comments — comment records linked to posts
- /albums — album records
- /photos — photo records with image URLs

### In Scope

✓ GET request analysis

✓ Response payload and structure review

✓ HTTP response header inspection

✓ Authentication and authorization verification

✓ Data-exposure and response-size observation

### Out of Scope

✗ Exploitation of any kind

✗ Authentication or authorization bypass

✗ Denial-of-service testing

✗ SQL injection or other injection testing

✗ Brute-force testing

✗ Any destructive or state-changing activity

## 4. Tools Used

The assessment relied on a small, standard toolset appropriate for a non-intrusive API review.

| Tool | Purpose |
|---|---|
| Postman | Composing and sending HTTP requests; inspecting responses and headers |
| Web Browser | Reviewing the public API documentation |
| JSONPlaceholder | The demonstration REST API under assessment |
| Canva | Report design and layout |
| GitHub | Version control and project repository |

## 5. Assessment Methodology

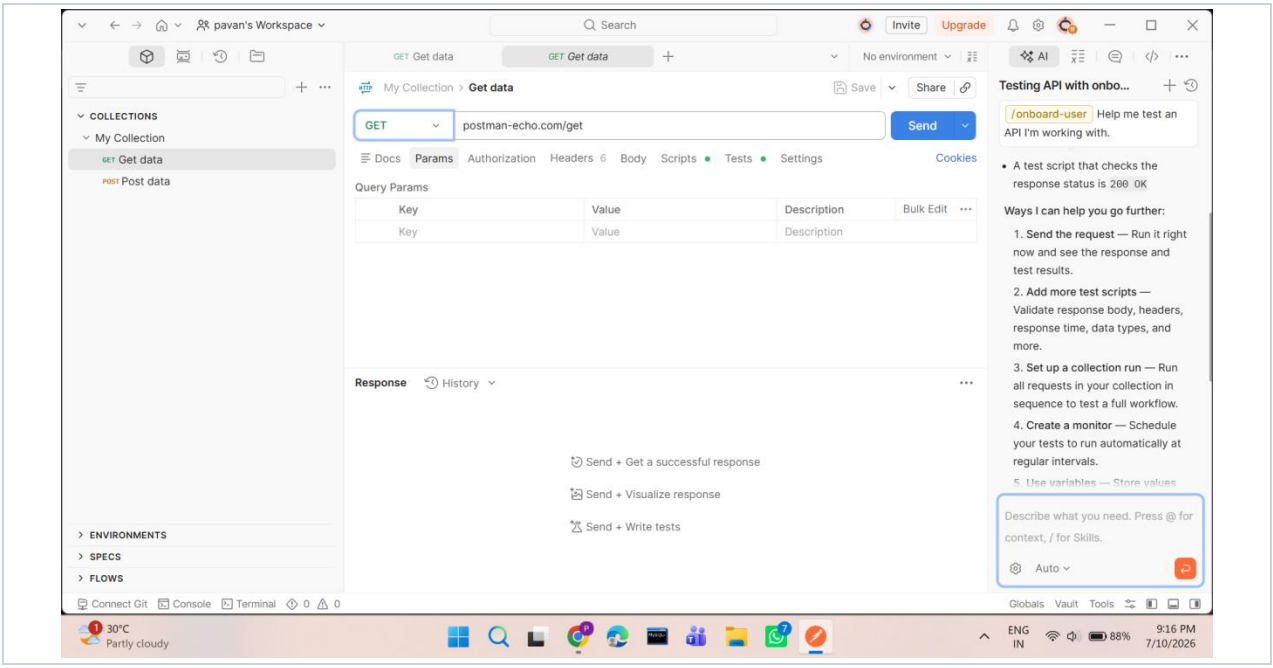
The API was assessed using a structured, read-only methodology. Each step built on the previous one, moving from documentation review through live request testing to risk documentation.

### Step 1 — Review the API documentation

The public documentation was reviewed first to understand the available resources, the expected request format, and the shape of the data each endpoint returns.

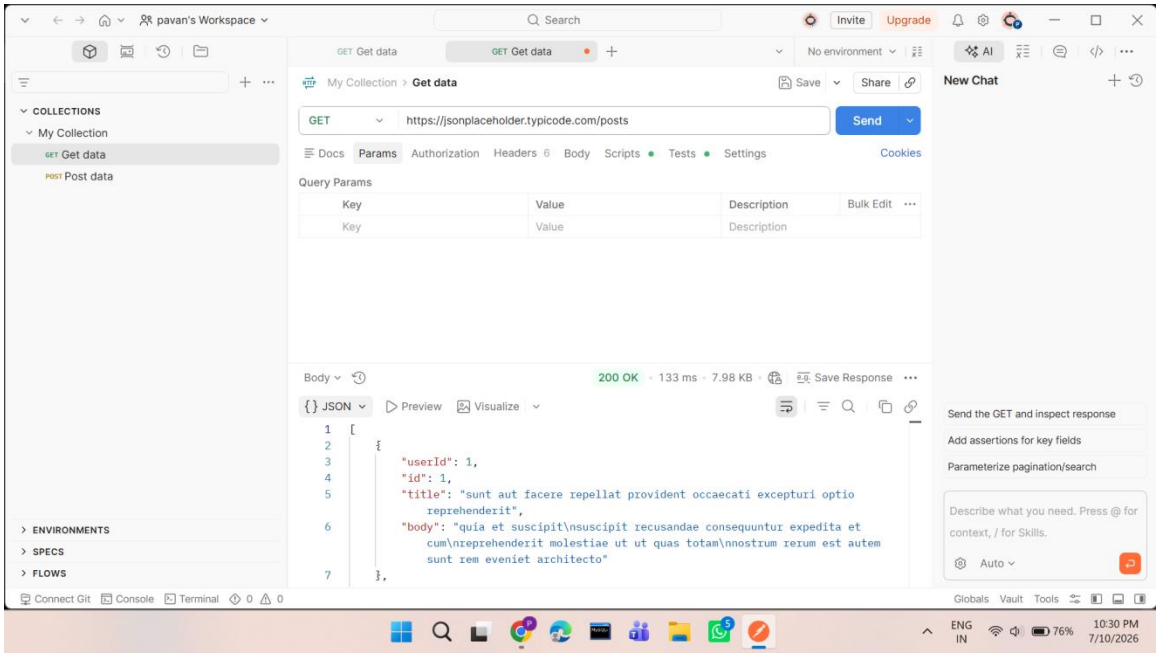
### Step 2 — Send HTTP GET requests using Postman

Postman was used as the request builder for the assessment. Each endpoint was called with a standard GET request so its live response could be captured and examined.



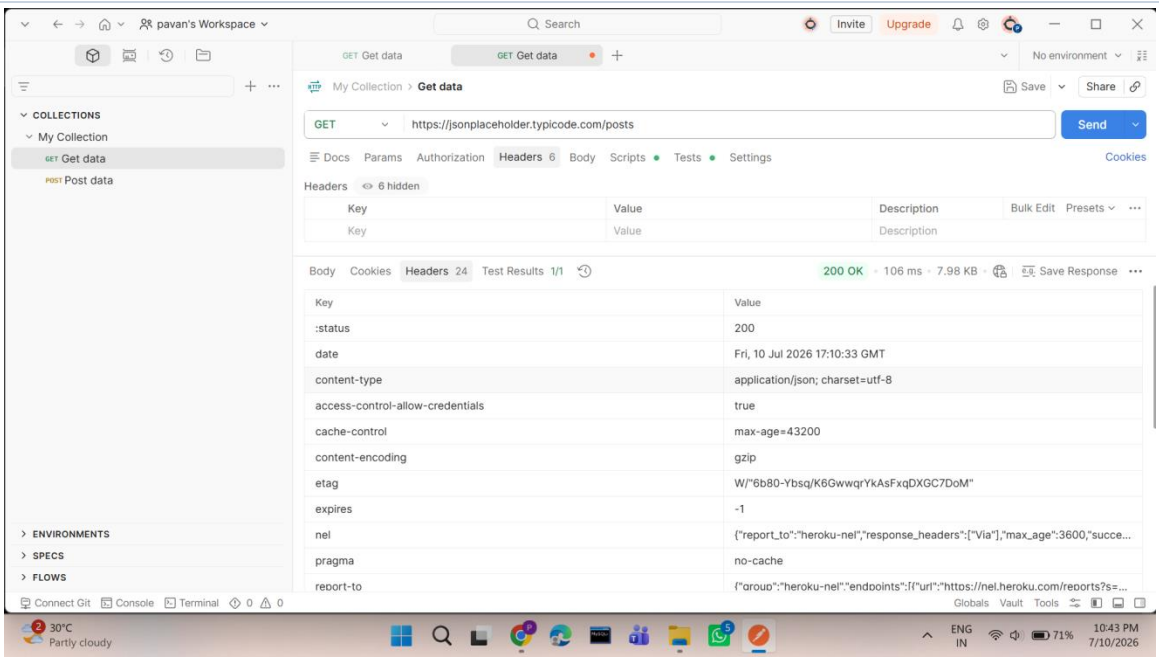

### Step 3 — Observe the API responses

Each response was reviewed for its status code, payload structure, size, and response time. The /posts endpoint returned a JSON array of post objects with a 200 OK status.



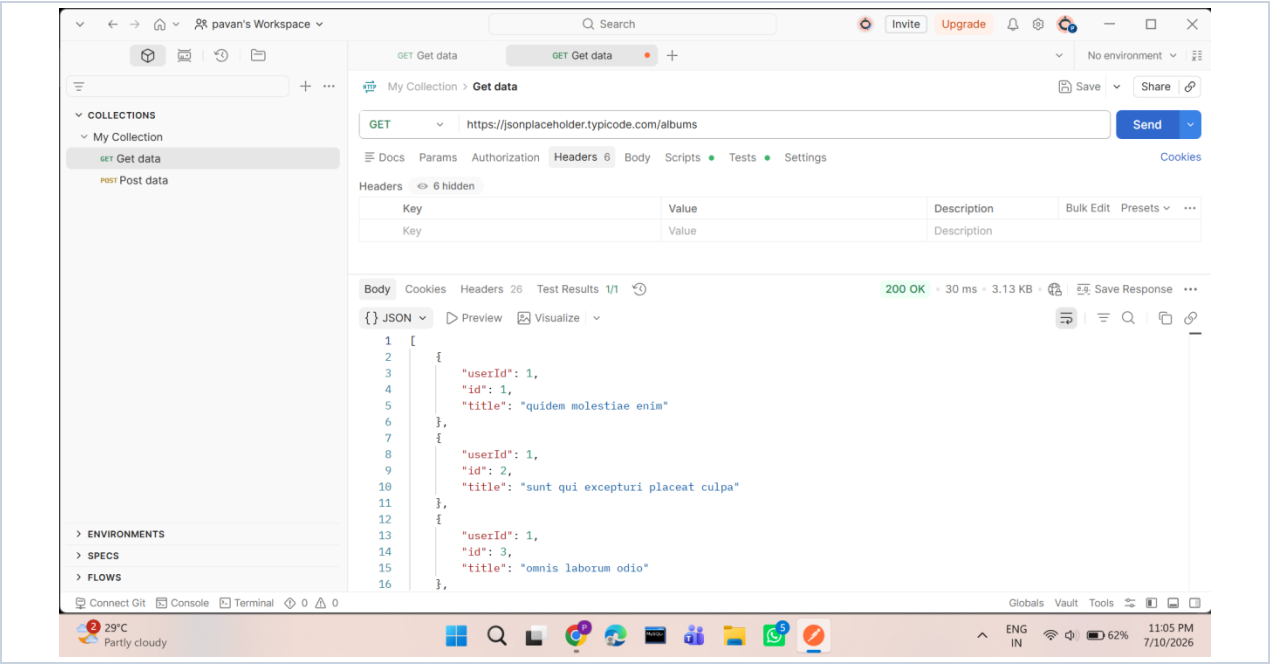

## Step 4 — Inspect the HTTP headers

The response headers were inspected to understand caching behaviour, content negotiation, compression, and — importantly — whether any security or authentication headers were present.



### Step 5 — Compare multiple endpoints for consistency

Several endpoints were compared to confirm that the API behaved consistently. The /albums endpoint, for example, returned a compact JSON array with no sensitive fields.




### Step 6 — Document risks and mitigations

Finally, the observations were consolidated into a risk assessment and a set of prioritised, best-practice recommendations, presented in the sections that follow.

## 6. API Response Analysis

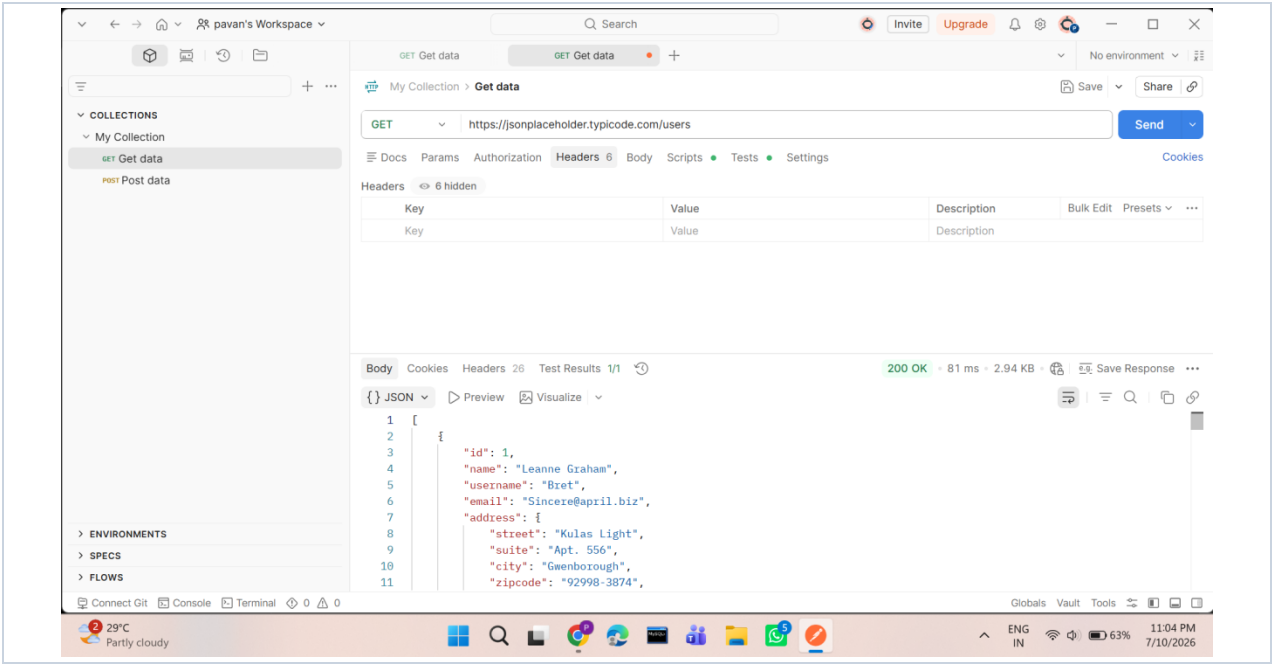
This section summarises the behaviour observed for each endpoint. All requests returned a 200 OK status and JSON-formatted data, and none required authentication.

### Endpoint: /posts

| Attribute | Observation |
|---|---|
| Status code | 200 OK |
| Authentication required | No |
| Response format | JSON array of post objects |
| Response size | ~7.98 KB (gzip compressed) |
| Observation | Publicly readable data returned without any credentials |

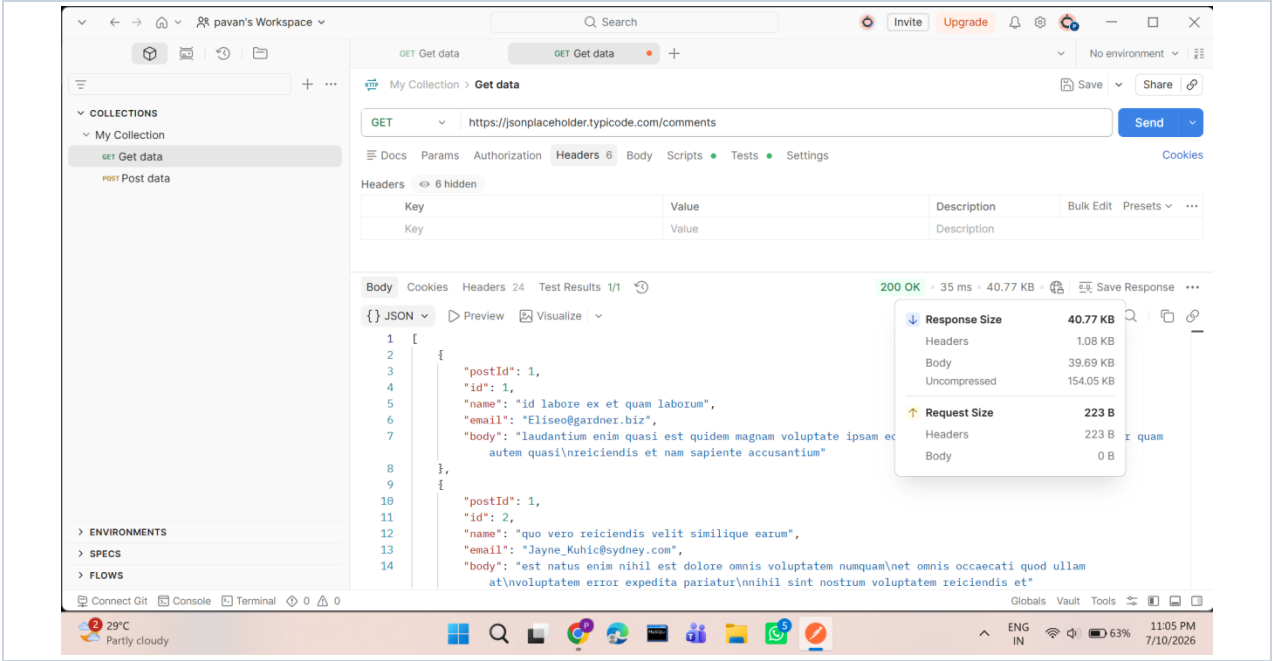
### Endpoint: /users

The /users endpoint returned user profile records including names, usernames, email addresses, physical addresses, and company details. While this is demonstration data, the same structure in a production API would represent a direct exposure of personally identifiable information (PII).




### Endpoint: /comments

The /comments endpoint returned a large collection of comment records without any pagination. The compressed response was 40.77 KB, but the uncompressed payload was roughly 154 KB — a meaningful amount of data to return in a single unbounded request.

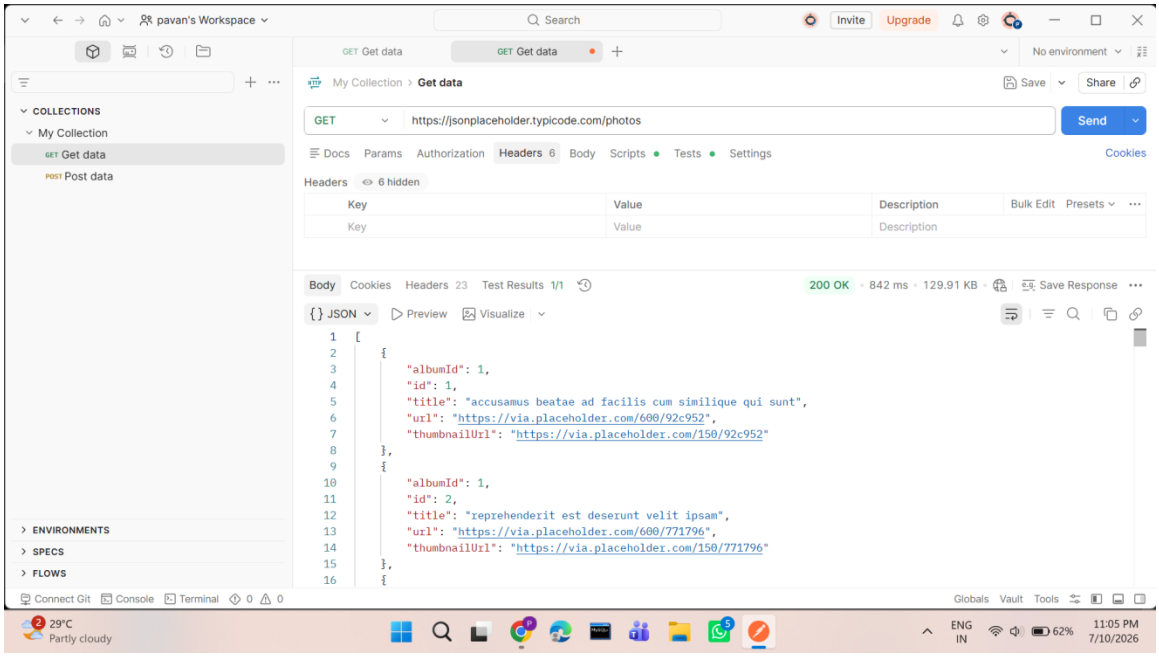



### Endpoint: /albums

The /albums endpoint returned album records containing only identifiers and titles. No sensitive information was observed, and the response was small and consistent with the other collections.

### Endpoint: /photos

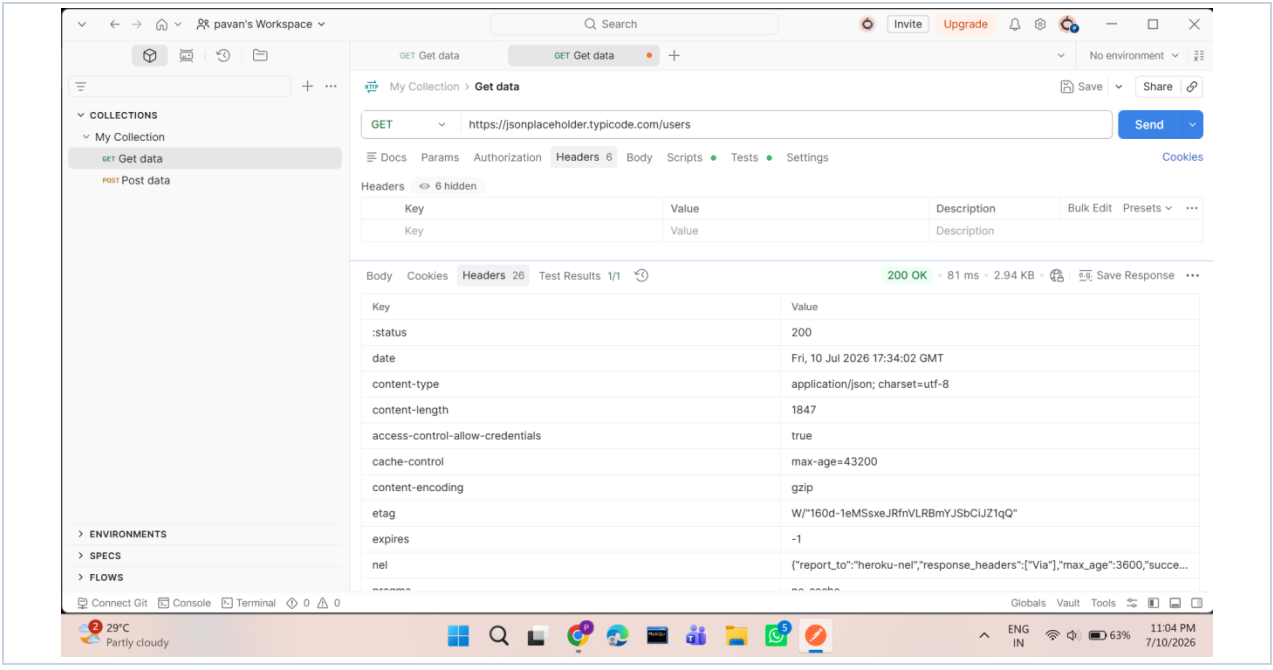
The /photos endpoint returned a large array of photo records, each containing image and thumbnail URLs. At roughly 130 KB, it was the largest response observed and reinforces the same pagination and response-sizing concern seen with /comments.


pavan's Workspace
Search
Invite
Upgrade
No environment
COLLECTIONS
My Collection
GET Get data
POST Post data
ENVIRONMENTS
SPECS
FLOWS
My Collection > Get data
Save
Share
GET
https://jsonplaceholder.typicode.com/photos
Send
Docs
Params
Authorization
Headers 6
Body
Scripts
Tests
Settings
Cookies
Headers 6 hidden
Key
Value
Description
Bulk Edit
Presets
Body
Cookies
Headers 23
Test Results 1/1
200 OK
842 ms
129.91 KB
Save Response
JSON
Preview
Visualize
[
{
"albumId": 1,
"id": 1,
"title": "accusamus beatae ad facilis cum similique qui sunt",
"url": "https://via.placeholder.com/600/92c952",
"thumbnailUrl": "https://via.placeholder.com/150/92c952"
},
{
"albumId": 1,
"id": 2,
"title": "reprehenderit est deserunt velit ipsam",
"url": "https://via.placeholder.com/600/771796",
"thumbnailUrl": "https://via.placeholder.com/150/771796"
},
{
Connect Git
Console
Terminal
Globals
Vault
Tools
29°C
Partly cloudy
ENG
IN
62%
11:05 PM
7/10/2026

## 7. HTTP Header Analysis

The response headers were reviewed across endpoints. The table below lists the notable headers observed and what each indicates about the API's behaviour.

| Header | Value observed | Purpose |
|---|---|---|
| content-type | application/json; charset=utf-8 | Declares the JSON response format |
| content-encoding | gzip | Confirms responses are compressed in transit |
| cache-control | max-age=43200 | Defines how long the response may be cached |
| etag | W/"..." | Resource fingerprint used for cache validation |
| expires / pragma | -1 / no-cache | Additional caching directives |
| date | Fri, 10 Jul 2026 … | Timestamp of the response |




### Security Observation

No authentication-related headers (such as Authorization challenges or WWW-Authenticate) were present, which is expected because the API is intentionally public for learning purposes. In a production API, the absence of authentication and hardening headers such as Strict-Transport-Security, X-Content-Type-Options, and a Content-Security-Policy would be treated as a finding to remediate.

## 8. Risk Assessment

The table below rates each observation. Severities are expressed for the demonstration context; the same findings would be re-rated substantially higher in a production system handling real user data.

| Risk | Severity | Observation |
|---|---|---|
| No authentication | Low | Expected for a demo API; would be High in production |
| Sensitive data exposure | Low | /usersreturns names, emails, and addresses — PII if real |
| No authorization | Low | No role or ownership checks on any resource |
| Large / unbounded responses | Low | /comments and /photos return large payloads without pagination |
| Rate limiting not verified | Informational | Could not be confirmed under read-only testing |

Overall, JSONPlaceholder behaves exactly as a public demonstration API is designed to. The value of this exercise is in recognising how each of these acceptable-in-a-demo behaviours would translate into a genuine security risk if the same design were shipped to production.

## 9. Recommendations

The following measures reflect standard API security best practice and map directly to the risks identified above.

1. Require authentication for any endpoint that exposes non-public data.
2. Enforce role-based, ownership-aware authorization on every resource.
3. Return only the fields a client needs, minimising unnecessary data exposure.
4. Apply pagination and sensible response-size limits to large collections.
5. Enable rate limiting and throttling to protect against abuse.
6. Validate and sanitise all client-supplied input.
7. Serve all traffic exclusively over HTTPS and add HSTS.
8. Log and monitor API requests to detect suspicious activity.
9. Follow the OWASP API Security Top 10 as a baseline for design and review.
10. Perform regular, repeatable API security assessments.

## 10. Conclusion

This assessment provided practical experience in understanding how REST APIs communicate through HTTP requests and responses. Working through a live API in Postman made the abstract concepts — status codes, payloads, headers, and compression — concrete and observable.

The exercise also demonstrated how a security analyst inspects an API without touching it destructively: reviewing documentation, sending controlled read requests, examining responses and headers, and reasoning about what the observed behaviour would mean in a real deployment.

Although JSONPlaceholder is intentionally designed as a public demonstration API, analysing it illustrated the core themes of API security — authentication, authorization, data exposure, response handling, and secure configuration. The result is a clearer, more practical understanding of how modern application security assessments are approached and documented.

## 11. References


1. OWASP API Security Top 10 — https://owasp.org/API-Security/
2. JSONPlaceholder — https://jsonplaceholder.typicode.com
3. Postman — https://www.postman.com
4. HTTP response status codes — MDN Web Docs — https://developer.mozilla.org
5. Future Interns — Cyber Security Internship, Task 3 documentation